

Come realizzare colori personalizzati [1]

Nell'installazione base di **R** sono disponibili 657 colori [1] che, all'interno delle funzioni che prevedono l'argomento grafico **col**, possono essere richiamati in alternativa con:

- **col**="nome del colore";
- **col**="codice esadecimale del colore".

Quindi possiamo, ad esempio, impiegare:

- per indicare il rosso sia **col**="red" sia **col**="#FF0000";
- per indicare il verde sia **col**="green" sia **col**="#00FF00";
- per indicare il blu sia **col**="blue" sia **col**="#0000FF".

Ma in **R** è anche possibile realizzare colori personalizzati.

Su uno schermo nero la sintesi additiva dei colori – secondo il modello RGB descritto nello *standard IEC 61966* – avviene regolando i rapporti di emissione di luce dei tre fosfori con i colori fondamentali rosso, verde e blu, la cui somma in parti uguali dà il bianco.

La sintesi RGB dei colori viene effettuata impiegando per ciascuno dei tre colori fondamentali 8 bit ovvero $2^8=256$ livelli di intensità. Il risultato è la disponibilità teorica di $256 \times 256 \times 256 = 16\,777\,216$ colori che come vediamo ora possono essere specificati in termini di intensità e di trasparenza del colore all'interno della funzione **rgb()**.

Il livello di intensità del colore può essere stabilito separatamente per rosso, verde e blu (in questo ordine) nella funzione **rgb()** in una delle due scale disponibili, specificate con l'argomento `maxColorValue` (valore di default `maxColorValue=1`):

- con `maxColorValue=1` si va da 0 a 1 per passi di $1/256=0.00390625$ – non è quindi pensabile di calcolare uno per uno i valori intermedi per cui in genere si riporta una frazione del colore, come 0.25 o 0.5 o 0.75 o altro, riducendo in pratica da 256 a 100 i livelli di intensità, ma esprimendoli in una scala che rende l'immediatezza numerica dei quantitativi di ciascun colore;
- con `maxColorValue=255` si va da 0 a 255 per passi di 1 – con questa scala si sfrutta in toto la graduazione della quantità desiderata di ciascun colore, ma si perde l'immediatezza numerica dei rapporti in cui sono stati miscelati.

La funzione **rgb()** prevede la possibilità di impostare anche un livello di trasparenza del colore (default = nessuna trasparenza), espresso nella stessa scala definita con `maxColorValue`.

La struttura della funzione è semplice, con gli argomenti relativi a colore e trasparenza che sono specificati come:

rgb(rosso, verde, blu, trasparenza, maxColorValue).

Questo esempio chiarisce quanto sopra più di qualsiasi cosa, copiate e incollate nella Console di R questo script e premete ↵ Invio.

```
# PERSONALIZZAZIONE DEI COLORI
```

```
# regolando il livello di intensità di rosso, verde e blu e la trasparenza
```

```
#
```

```
mydata <- rnorm(5000, mean=0, sd=1) # genera cinquemila valori con media=0, ds=1
```

```
#
```

```
windows() # apre e inizializza una nuova finestra grafica
```

```
par(mfrow=c(4,4)) # suddivisione la finestra grafica in 4 righe per 4 colonne (16 quadranti)
```

```
#
```

```
# colore al 100% senza trasparenza colore, scala da 0 a 255
```

```
#
```

```
hist(mydata, col=rgb(255, 0, 0, 255, maxColorValue=255)) # rosso
```

```
hist(mydata, col=rgb(0, 255, 0, 255, maxColorValue=255)) # verde
```

```
hist(mydata, col=rgb(0, 0, 255, 255, maxColorValue=255)) # blu
```

```
hist(mydata, col=rgb(255, 255, 0, 255, maxColorValue=255)) # rosso + verde
```

```
hist(mydata, col=rgb(255, 0, 255, 255, maxColorValue=255)) # rosso + blu
```

```
hist(mydata, col=rgb(0, 255, 255, 255, maxColorValue=255)) # verde + blu
```

```
hist(mydata, col=rgb(255, 255, 255, 255, maxColorValue=255)) # rosso + verde + blu =  
bianco
```

```
hist(mydata, col=rgb(0, 0, 0, 255, maxColorValue=255)) # nessun colore = nero
```

```
#
```

```
# colore al 100% con trasparenza del colore al 50%, scala da 0 a 1
```

```
#
```

```
hist(mydata, col=rgb(1, 0, 0, 0.5, maxColorValue=1)) # rosso
```

```
hist(mydata, col=rgb(0, 1, 0, 0.5, maxColorValue=1)) # verde
```

```
hist(mydata, col=rgb(0, 0, 1, 0.5, maxColorValue=1)) # blu
```

```
hist(mydata, col=rgb(1, 1, 0, 0.5, maxColorValue=1)) # rosso + verde
```

```
hist(mydata, col=rgb(1, 0, 1, 0.5, maxColorValue=1)) # rosso + blu
```

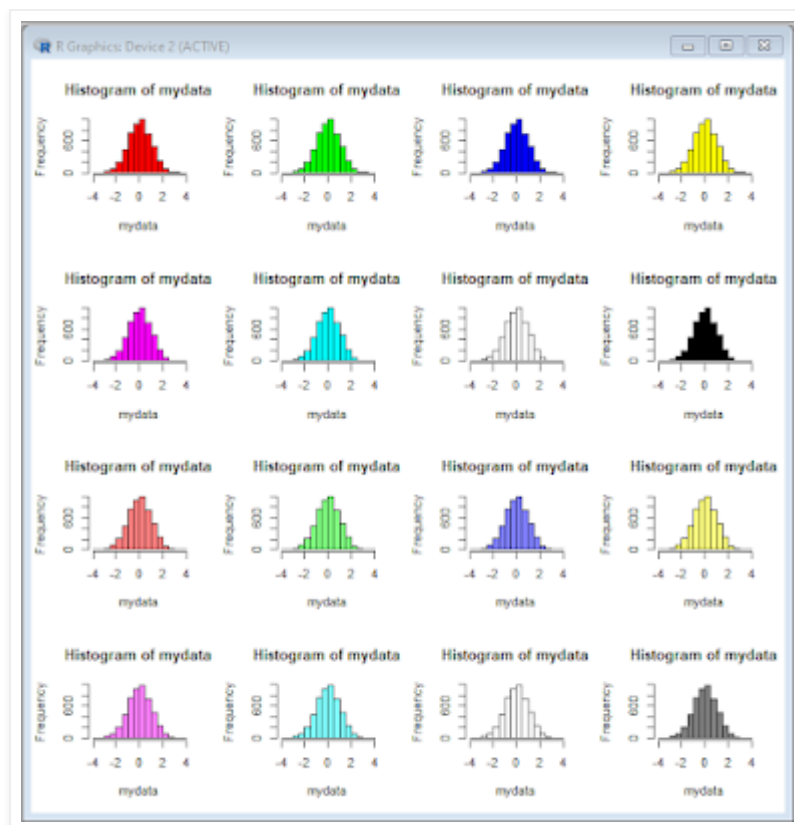
```
hist(mydata, col=rgb(0, 1, 1, 0.5, maxColorValue=1)) # verde + blu
```

```
hist(mydata, col=rgb(1, 1, 1, 0.5, maxColorValue=1)) # rosso + verde + blu + trasparenza =  
bianco
```

```
hist(mydata, col=rgb(0, 0, 0, 0.5, maxColorValue=1)) # nessun colore + trasparenza = grigio
```

```
#
```

Questi sono i colori risultanti.



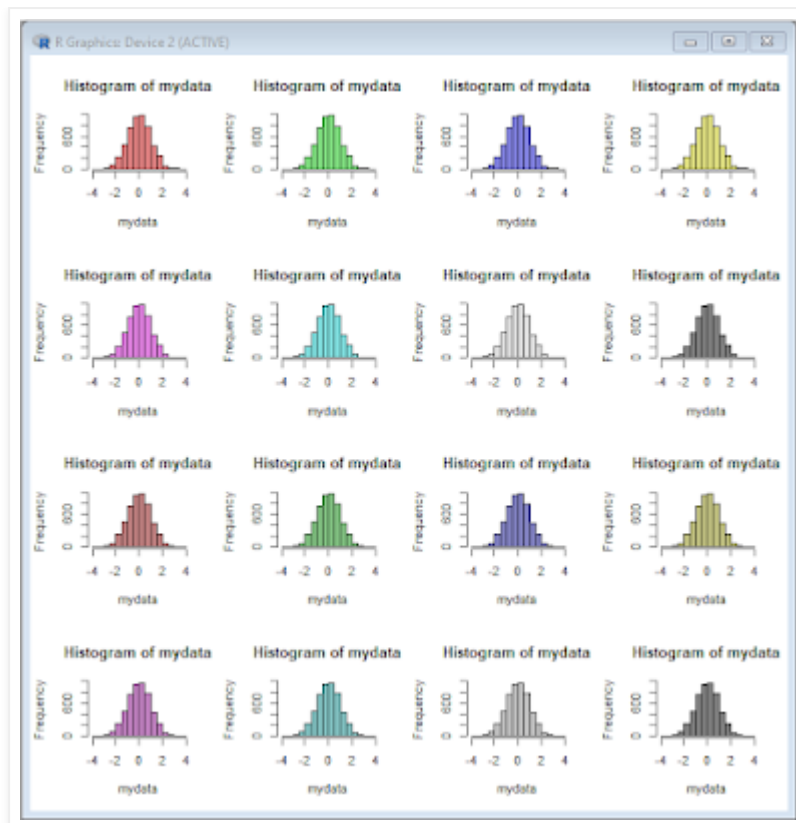
Ed ecco un secondo esempio, copiate e incollate nella Console di R questo script e premete ↵ Invio.

```

# PERSONALIZZAZIONE DEI COLORI
# regolando il livello di intensità di rosso, verde e blu e la trasparenza
#
mydata <- rnorm(5000, mean=0, sd=1) # genera cinquemila valori con media=0, ds=1
#
windows() # apre e inizializza una nuova finestra grafica
par(mfrow=c(4,4)) # suddivisione la finestra grafica in 4 righe per 4 colonne (16 quadranti)
#
# colore al 75% e trasparenza al 50%, scala da 0 a 255
#
hist(mydata, col=rgb(191, 0, 0, 127, maxColorValue=255)) # rosso
hist(mydata, col=rgb(0, 191, 0, 127, maxColorValue=255)) # verde
hist(mydata, col=rgb(0, 0, 191, 127, maxColorValue=255)) # blu
hist(mydata, col=rgb(191, 191, 0, 127, maxColorValue=255)) # rosso + verde
hist(mydata, col=rgb(191, 0, 191, 127, maxColorValue=255)) # rosso + blu
hist(mydata, col=rgb(0, 191, 191, 127, maxColorValue=255)) # verde + blu
hist(mydata, col=rgb(191, 191, 191, 127, maxColorValue=255)) # rosso + verde + blu +
trasparenza = grigio
hist(mydata, col=rgb(0, 0, 0, 127, maxColorValue=255)) # nessun colore + trasparenza =
grigio
#
# colore al 50% e trasparenza al 50%, scala da 0 a 1
#
hist(mydata, col=rgb(0.5, 0, 0, 0.5, maxColorValue=1)) # rosso
hist(mydata, col=rgb(0, .5, 0, 0.5, maxColorValue=1)) # verde
hist(mydata, col=rgb(0, 0, 0.5, 0.5, maxColorValue=1)) # blu
hist(mydata, col=rgb(0.5, 0.5, 0, 0.5, maxColorValue=1)) # rosso + verde
hist(mydata, col=rgb(0.5, 0, 0.5, 0.5, maxColorValue=1)) # rosso + blu
hist(mydata, col=rgb(0, 0.5, 0.5, 0.5, maxColorValue=1)) # verde + blu
hist(mydata, col=rgb(0.5, 0.5, 0.5, 0.5, maxColorValue=1)) # rosso + verde + blu +
trasparenza = grigio
hist(mydata, col=rgb(0, 0, 0, 0.5, maxColorValue=1)) # nessun colore + trasparenza = grigio
#

```

Questi sono i colori risultanti.



Trovare il codice esadecimale corrispondente ad uno dei 256 valori decimali possibili è semplice, perché la funzione **rgb()** lo restituisce direttamente, quindi ad esempio se eseguite questo codice

```
# converte colore al 100% senza trasparenza in esadecimale
```

```
#
```

```
rgb(255, 0, 0, maxColorValue=255) # rosso
```

```
rgb(0, 255, 0, maxColorValue=255) # verde
```

```
rgb(0, 0, 255, maxColorValue=255) # blu
```

```
#
```

nella Console di R compaiono i codici esadecimali dei tre colori base espressi al 100% di intensità:

```
> # converte colore al 100% senza trasparenza in esadecimale
```

```
> #
```

```
> rgb(255, 0, 0, maxColorValue=255) # rosso
```

```
[1] "#FF0000"
```

```
> rgb(0, 255, 0, maxColorValue=255) # verde
```

```
[1] "#00FF00"
```

```
> rgb(0, 0, 255, maxColorValue=255) # blu
```

```
[1] "#0000FF"
```

Quindi in definitiva se in qualche elemento di una funzione grafica volete impiegare il colore rosso potete riportare in alternativa una di queste espressioni:

→ **col="red"** cioè la denominazione del colore riportata nella tabella dei colori base di **R** [1];

→ **col=rgb(255, 0, 0, maxColorValue=255)** cioè il colore realizzato specificando l'intensità di ciascuno dei tre colori fondamentali in decimale con la funzione **rgb()**;

→ **col="#FF0000"** cioè specificando l'intensità di ciascuno dei tre colori fondamentali con il codice esadecimale generato dalla funzione **rgb()**.

Infine se eseguite questo codice, nel quale le intensità del colore sono state poste al 75% (**0.75**),

```
# converte colore al 75% con trasparenza al 50% in esadecimale
#
rgb(0.75, 0, 0, 0.5, maxColorValue=1) # rosso
rgb(0, 0.75, 0, 0.5, maxColorValue=1) # verde
rgb(0, 0, 0.75, 0.5, maxColorValue=1) # blu
#
```

vedete che riporta nella Console di R il valore esadecimale generato dalla funzione **rgb()** aggiungendo una trasparenza del colore uguale al 50% (**0.5**):

```
> # converte colore al 75% con trasparenza al 50%, da decimale a esadecimale
> #
> rgb(0.75, 0, 0, 0.5, maxColorValue=1) # rosso
[1] "#BF000080"
> rgb(0, 0.75, 0, 0.5, maxColorValue=1) # verde
[1] "#00BF0080"
> rgb(0, 0, 0.75, 0.5, maxColorValue=1) # blu
[1] "#0000BF80"
```

Quindi se in qualche elemento di una funzione grafica volete impiegare ad esempio il colore verde ma con un livello di intensità del 75% e con un livello di trasparenza del 50% potete riportare in alternativa:

→ **col=rgb(0, 0.75, 0, 0.5, maxColorValue=1)** oppure molto più sinteticamente
→ **col="#00BF0080"**

Conclusione: con la funzione **rgb()** è possibile al bisogno definire un proprio colore personalizzato miscelando i tre colori fondamentali rosso, verde e blu e scegliendone anche il livello di trasparenza. Trovate il seguito con l'illustrazione delle altre due funzioni che consentono di realizzare colori personalizzati nel post [Come realizzare colori personalizzati \[2\]](#).

[1] Vedere il post [Nomi e codici dei colori di R](#).
